LECTURER: MAJA POPOVIĆ

REINFORCEMENT LEARNING

REINFORCEMENT LEARNING
TOPIC OUTLINE

Introduction to Reinforcement Learning	1
Dynamic Programming and Markov Decision Processes	2
Bandit Problems	3
Q-Learning	4
Reinforcement Learning Approaches	5
Summary: Reinforcement Learning	6

UNIT 2

DYNAMIC PROGRAMMING AND MARKOV DECISION PROCESSES

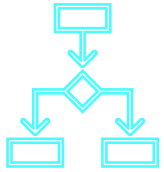
Recommended sources:

- https://www.davidsilver.uk/teaching/

  on-line lectures (videos + slides)

- https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf

  on-line book



Sequential Decision Process (SDP)

- **state space:** set of possible states
- **action space:** set of available actions
- **state-action space:** combined states and actions
- **reward function:** rewards for actions in states
- **transition function:** state changes



SDP comprises stages with costs and actions, shaping future outcomes via state transitions and decision-making

Source of the text: Bellman, 1957.

Markov decision process

Markov state

Markov state

- only the current state is relevant for future states
- the rest of the history can be ignored

state in general:

$$P(S_{t+1}|H_t) = P(S_{t+1}|S_t, S_{t-1}, S_{t-2}, \dots S_2, S_1)$$

Markov state:

$$P(S_{t+1}|H_t) = P(S_{t+1}|S_t, S_{t-1}, S_{t-2}, \dots S_2, S_1) = P(S_{t+1}|S_t)$$

Markov state

a state is Markov if and only if:

$$P(S_{t+1}|H_t) = P(S_{t+1}|S_t, S_{t-1}, S_{t-2}, \dots S_2, S_1) = P(S_{t+1}|S_t)$$

state transition probability (from state s to successor state s'):

$$P_{ss'} = P(S_{t+1} = s' | \, S_t = s)$$

Markov states: transition probability matrix

state transition probability

(from each of n possible states s to each of n possible states s')

$$P = \begin{bmatrix} P_{11} & \cdots & P_{1n} \\ \vdots & \ddots & \vdots \\ P_{n1} & \cdots & P_{nn} \end{bmatrix}$$

each row sums to 1

Markov process

Markov process (MP)

- a sequence of random Markov states $S_1 S_2 \ldots S_t \ldots$

- a tuple (S, P)
  - $S =$ a (finite) set of states
  - $P =$ transition probability matrix

$$P_{ss'} = P(S_{t+1} = s' \mid S_t = s)$$

Markov reward process (MRP)

- a sequence of random Markov states $S_1 S_2 \ldots S_t \ldots$ with reward values

- a tuple (S, P, R, β)
  - S = a finite set of states
  - P = transition probability matrix

$$P_{ss'} = P(S_{t+1} = s' \mid S_t = s)$$

  - R = a (immediate) reward function

$$R_s = E[R_{t+1} \mid S_t = s]$$

  - β = a discount factor, $\beta \in [0,1]$

State value function for MRPs

long-term value of state s

$$v(s) = E[R_{t+1} + \beta R_{t+2} + \beta^2 R_{t+3} + \cdots \mid S_t = s]$$

$$v(s) = E[G_t \mid S_t = s]$$

expected total future reward (cumulative reward, return) G_t starting from state s

Bellman equation for MRPs

$$
\begin{aligned}
v(s) &= E[G_t \mid S_t = s] \\
&= E[R_{t+1} + \beta R_{t+2} + \beta^2 R_{t+3} + \beta^3 R_{t+4} + \ \ldots \mid S_t = s] \\
&= E[R_{t+1} + \beta(R_{t+2} + \beta R_{t+3} + \beta^2 R_{t+4}) + \ \ldots) \mid S_t = s] \\
&= E[R_{t+1} + \beta G_{t+1} \mid S_t = s] \\
&= E[R_{t+1} + \beta v(S_{t+1}) \mid S_t = s]
\end{aligned}
$$

value function decomposed into two components:

- immediate reward R_{t+1}
- (discounted value) function of successor state $\beta v(S_{t+1})$

Bellman equation for MRPs

$$v(s) = E[R_{t+1} + \beta v(S_{t+1}) \mid S_t = s]$$

$$v(s) = R_s + \beta \sum_{s' \in S} P_{ss'} \cdot v(s')$$

$$E[R_{t+1} \mid S_t = s] = R_s$$

Bellman equation for MRPs

$$v(s) = R_s + \beta \sum_{s' \in S} P_{ss'} \cdot v(s')$$

matrix form:

$$v = R + \beta P v$$

$$\begin{bmatrix} v(1) \\ \vdots \\ v(n) \end{bmatrix} = \begin{bmatrix} R_1 \\ \vdots \\ R_n \end{bmatrix} + \beta \begin{bmatrix} P_{11} & \cdots & P_{1n} \\ \vdots & \ddots & \vdots \\ P_{n1} & \cdots & P_{nn} \end{bmatrix} \begin{bmatrix} v(1) \\ \vdots \\ v(n) \end{bmatrix}$$

Bellman equation for MRPs

$$v = R + \beta P v$$

a linear equation

$$v - \beta P v = R$$

$$v = (1 - \beta P)^{-1} R$$

- solving complexity $O(n^3)$
- direct solution only possible for small processes
- many iterative methods for large ones:
  - dynamic programming
  - Monte-Carlo method
  - temporal difference learning

Markov decision process

Markov decision process (MDP)

reinforcement learning environment fully observable $(S_t^A = S_t^E)$

$\Rightarrow$ described by Markov decision process

almost all reinforcement learning problems can be formalised as MDP

- partially observable problems can be converted into MDPs
- bandits are MDPs with one state

Markov decision process (MDP)

- a tuple (S, A, P, R, β)
  - S = a finite set of states
  - A = a finite set of actions
  - P = transition probability matrix

$$P_{ss'}^{a} = P(S_{t+1} = s' \mid S_t = s,\ A_t = a)$$

  - R = a (immediate) reward function

$$R_{s}^{a} = E[R_{t+1} \mid S_t = s,\ A_t = a]$$

  - β = a discount factor, $\beta \in [0,1]$

Markov decision process

Policy

distribution of actions given states

$$\pi(a|s) = P(A_t = a \mid S_t = s)$$

$$\sum_{a \in A} \pi(a|s) = 1$$

Markov decision process

Given a Markov decision process (S, A, P, R, β) and a policy π:

- the sequence of states is a Markov process (S, P^{π})
- the sequence of states and rewards is a Markov reward process $(S, P^{\pi}, R^{\pi}, \gamma)$

$$P^{\pi}_{ss'} = \sum_{a \in A} \pi(a|s) \cdot P^{a}_{ss'}$$

$$R^{\pi}_{s} = \sum_{a \in A} \pi(a|s) \cdot R^{a}_{s}$$

Markov decision process

Value function

state value function:

$$v_\pi(s) = E_\pi[G_t \mid S_t = s]$$

expected return starting from the state s and then following the policy π

action value function:

$$q_\pi(s, a) = E_\pi[G_t \mid S_t = s, A_t = a]$$

expected return starting from the state s, taking the action a, and then following the policy π

Bellman equation for MDPs

expectation equation

state value function:

$$v_\pi(s) = E_\pi[R_{t+1} + \beta v_\pi(S_{t+1}) \mid S_t = s]$$

action value function:

$$q_\pi(s, a) = E_\pi[R_{t+1} + \beta q_\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a]$$

Bellman expectation equation for $v_\pi(s)$ (#1)

$$v_\pi(s) = E_\pi[R_{t+1} + \beta v_\pi(S_{t+1}) \mid S_t = s]$$

$$v_\pi(s) = \sum_{a \in A} \pi(a|s) \cdot q_\pi(s, a)$$

Bellman expectation equation for $q_\pi(s)$ (#1)

$$q_\pi(s,a) = E_\pi[R_{t+1} + \beta q_\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a]$$

$$q_\pi(s,a) = R_s^a + \beta \sum_{s' \in S} P_{ss'}^a \cdot v_\pi(s')$$

Bellman expectation equation for $v_\pi(s)$ (#2)

$$v_\pi(s) = \sum_{a \in A} \pi(a|s) \cdot q_\pi(s,a)$$

$$v_\pi(s) = \sum_{a \in A} \pi(a|s) \cdot \left(R_s^a + \beta \sum_{s' \in S} P_{ss'}^a \cdot v_\pi(s') \right)$$

Bellman expectation equation for $q_\pi(s)$ (#2)

$$q_\pi(s,a) = R_s^a + \beta \sum_{s' \in S} P_{ss'}^a \cdot v_\pi(s')$$

$$q_\pi(s,a) = R_s^a + \beta \sum_{s' \in S} P_{ss'}^a \sum_{a \in A} \pi(a'|s') \cdot q_\pi(s',a')$$

Optimal value function

$$v_*(s) = \max_\pi v_\pi(s)$$

$$q_*(s,a) = \max_\pi q_\pi(s,a)$$

the maximal value function over all policies π

- specifies the best performance in a Markov decision process
  - a MDP is “solved” if the optimal value function is known

Bellman optimality equation for v_* (#1)

$$v_*(s) = \max_a q_*(s, a)$$

Bellman optimality equation for q_* (#1)

$$q_*(s,a) = R_s^a + \beta \sum_{s' \in S} P_{ss'}^a \cdot v_*(s')$$

Bellman optimality equation for v_* (#2)

$$v_*(s) = \max_a q_*(s,a)$$

$$v_*(s) = \max_a \left(R_s^a + \beta \sum_{s' \in S} P_{ss'}^a \cdot v_*(s') \right)$$

Bellman optimality equation for q_* (#2)

$$q_*(s,a) = R_s^a + \beta \sum_{s' \in S} P_{ss'}^a \cdot v_*(s')$$

$$q_*(s,a) = R_s^a + \beta \sum_{s' \in S} P_{ss'}^a \cdot \max_{a'} q_*(s',a')$$

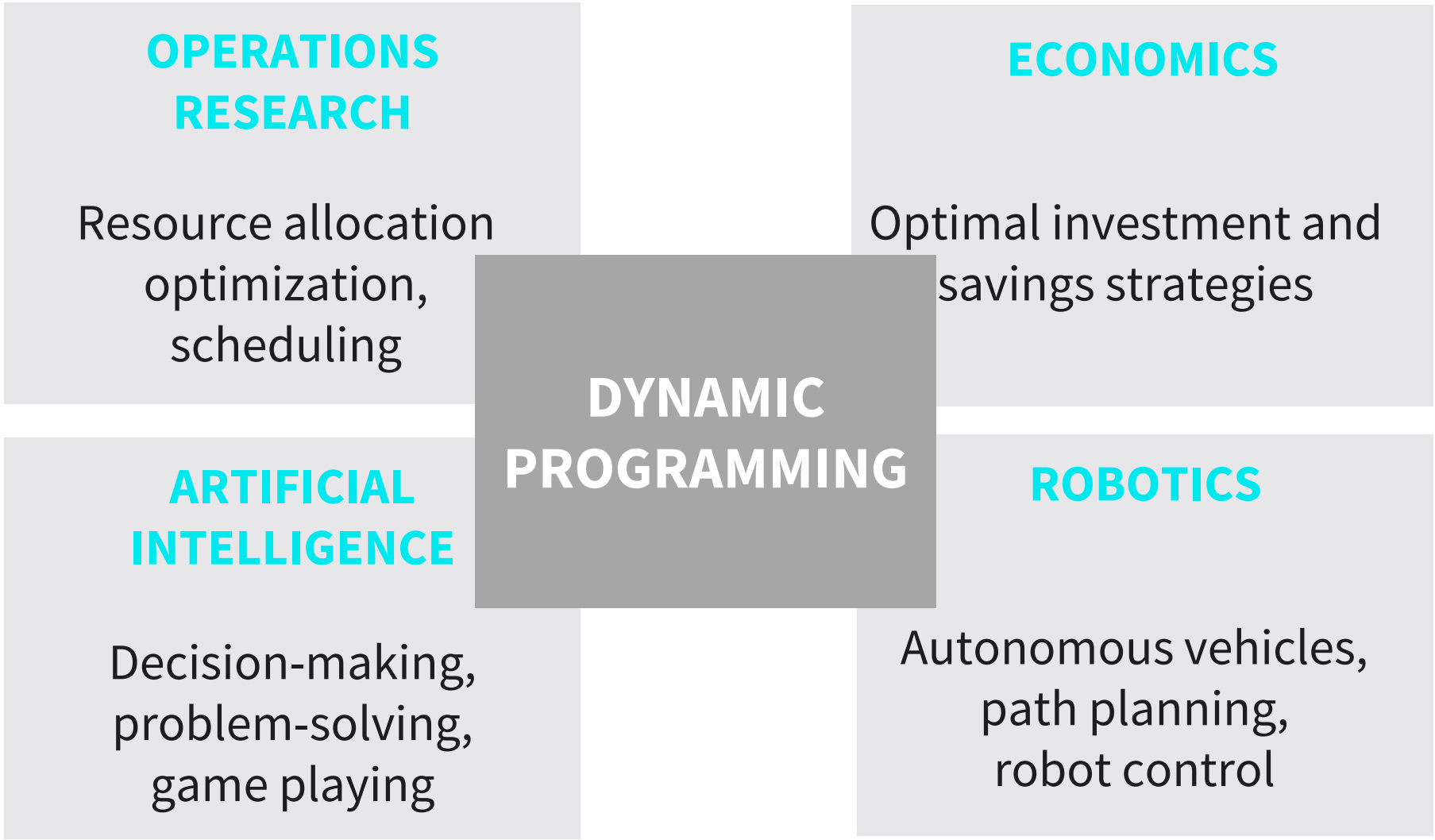
Bellman optimality equation

- non-linear
- no closed-form solution

- many iterative solution methods:
  - value iteration
  - policy iteration
  - Q-learning
  - SARSA

Dynamic programming

APPLICATIONS OF DYNAMIC PROGRAMMING



Source of the text: Adda, Cooper, & Cooper 2003; Sutton and Barto, 2018; Xue, Li, & Yang, 2019.

Enumeration Strategy

Exhaustively list, evaluate, and compare all possible routes

- Count all possible routes
- Calculate total distance for each route
- Choose shortest route



Time increases exponentially

$$N_{route} = n_1 \cdot n_2 \cdot \ldots \cdot n_k$$

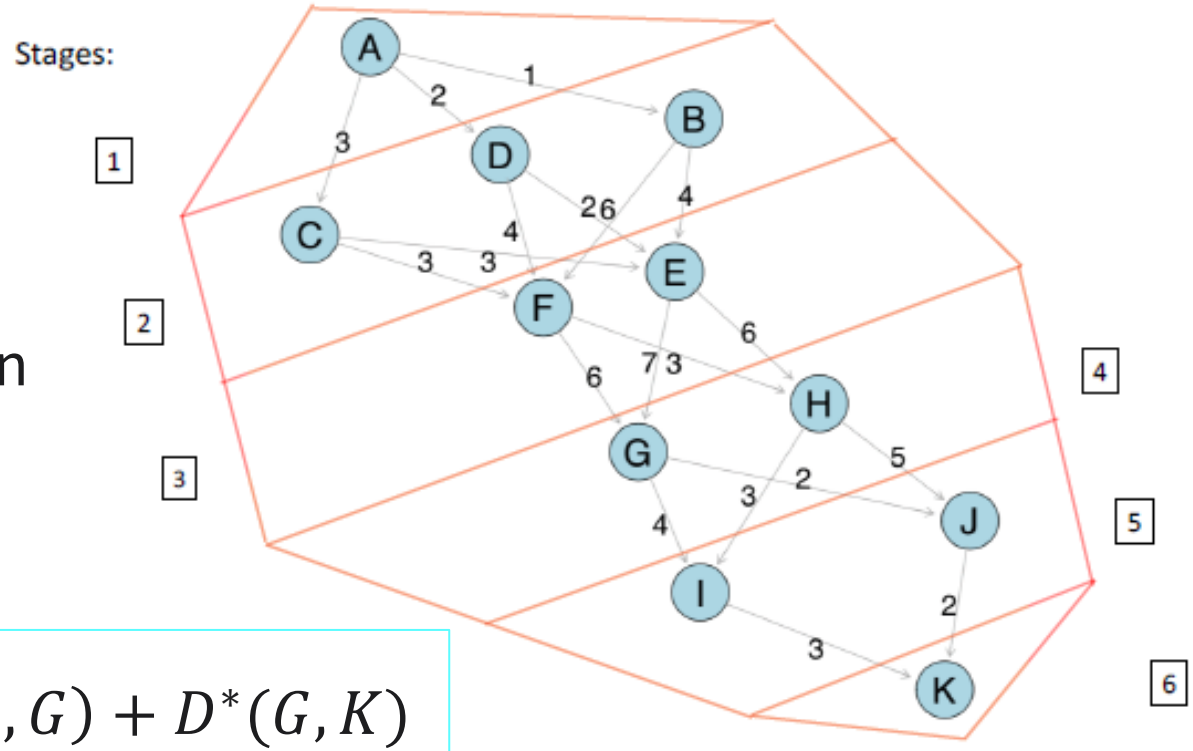


Can complexity be reduced?

Divide and Conquer Strategy

Decompose into independent sub-problems for efficient optimization

- Decompose into route segments
- Minimize segments independently
- Consider minimum distance between nodes in distinct stages



$$D^*(A, K \mid G) = D^*(A, G) + D^*(G, K)$$

Source of the image: Own representation.

Dynamic programming

- A method for solving complex problems
  by breaking them into sub-problems
    solve sub-problems and combine the solutions

- suitable for solving MDPs

Dynamic Programming

To be solvable by DP, a complex problem should have two properties:

1. optimal solution can be decomposed into sub-problems
2. sub-problems recur many times, can be stored and reused

MDP have both:

1. Bellman equation provides decomposition
2. solutions of value function are stored and reused

Prediction and Control

- Prediction = policy evaluation
  - **evaluate** the future reward for a **given policy**

- Control = policy improvement
  - **optimise** the future reward by **finding the best policy**

Dynamic Programming: planning in an MDP

- prediction (policy evaluation)
  - input: MDP (S, A, P, R, β) and policy π
    or: MRP $(S, P^{\pi}, R^{\pi}, \beta)$
  - output: value function v_{π}

- control (policy improvement)
  - input: MDP (S, A, P, R, β)
  - output: optimal value function v_*
    and optimal policy π_*

Policy evaluation

Iterative policy evaluation

- for each iteration $k + 1$:
  - for all states $s \in S$:
    - calculate $v_{k+1}(s)$ using $v_k(s')$

$$v_1 \rightarrow v_2 \rightarrow \cdots \rightarrow v_\pi$$

s' = successor state of s

Iterative policy evaluation

$$v_{k+1}(s) = \sum_{a \in A} \pi(a|s) \cdot \left(R_s^a + \beta \sum_{s' \in S} P_{ss'}^a \cdot v_k(s') \right)$$

$$v_{\pi}(s) = \sum_{a \in A} \pi(a|s) \cdot \left(R_s^a + \beta \sum_{s' \in S} P_{ss'}^a \cdot v_{\pi}(s') \right)$$

Policy improvement

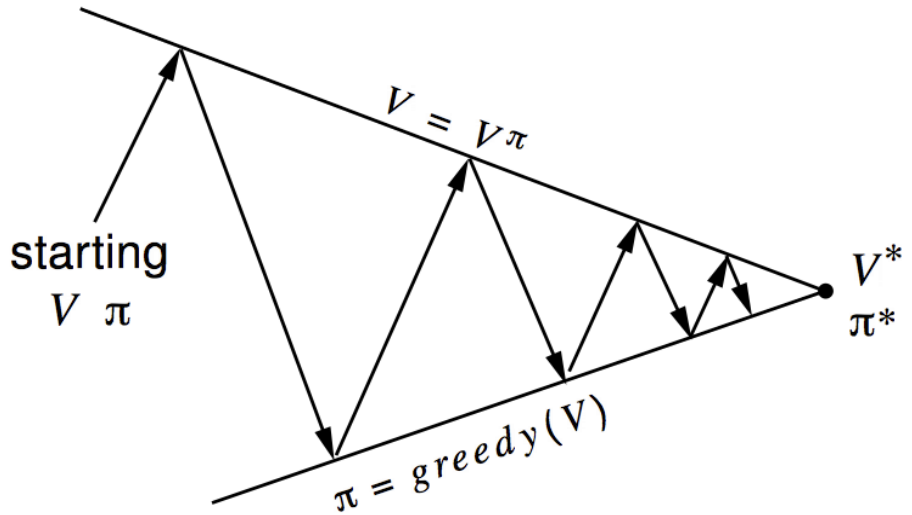
Policy iteration

Policy iteration

- evaluate a given policy
- improve the policy greedily

$$\pi' = \underset{a \in A}{\operatorname{argmax}}\, q_\pi(s, a)$$

- repeat



- eventually, optimal policy π_* and optimal value v_* are reached

Policy iteration

greedy strategy improves the value

$$\pi' = \underset{a \in A}{\operatorname{argmax}}\, q_\pi(s, a)$$

$$v_{\pi'}(s) = q_\pi\big(s, \pi'(s)\big) = \max_{a \in A} q_\pi(s, a) \geq q_\pi\big(s, \pi(s)\big) = v_\pi(s)$$

Modified policy iteration

stop before the value converges all the way to v_π

- introduce a stopping condition
  e.g. ε-convergence of value function

- stop after k iterations

Value iteration

Principle of optimality

a policy $\pi(a|s)$ achieves the optimal value $v_*(s)$ from state s if and only if

for any reachable successor state s':

$$v_\pi(s') = v_*(s')$$

Value iteration

if we know $v_*(s')$,
$v_*(s)$ can be found by iterative updating

$$v_*(s) = \max_a \left(R_s^a + \beta \sum_{s' \in S} P_{ss'}^a \cdot v_*(s') \right)$$

start with final rewards and look backwards

Value iteration

- for each iteration $k + 1$:
  - for all states $s \in S$:
    - calculate $v_{k+1}(s)$ using $v_k(s')$

  Without an explicit policy (unlike policy iteration)

  intermediate values may not correspond to any policy

  $v_1 \rightarrow v_2 \rightarrow \cdots \rightarrow v_*$

s' = successor state of s

Value iteration

$$v_{k+1}(s) = \max_{a \in A} \left(R_s^a + \beta \sum_{s' \in S} P_{ss'}^a \cdot v_k(s') \right)$$

ALGORITHMS FOR THE OPTIMAL CONTROL OF THE MDP

Value Iteration
Iterative Optimisation

- Initialise values for each state
- Update values using Bellman's equation
- Calculate new values based on maximal reward
- Converge to optimal value function

Policy Iteration
The Actor – Critic Method

- Policy evaluation **(Critic)**
- Policy improvement **(Actor)**
- Terminate when optimal policy is found

Source of the text: Sutton and Barto, 2018.

Dynamic programming: overview

problem	Bellman equation	Algorithm
prediction	Bellman expectation equation	policy evaluation
control	Bellman expectation equation + greedy policy improvement	policy iteration
control	Bellman optimality equation	value iteration

Questions?

REVIEW STUDY GOALS



- Define and structure **sequential decisions problems**
- Understand and apply **optimal rules** for effective decision-making in sequential processes
- Learn how to implement **optimization principles** practically through computer programs and applications
- Explore the significance of **utility theory** in various applications

SESSION 2

TRANSFER TASK



Case study



A navigation app aims to enhance travel efficiency by finding the shortest route while minimizing travel time and fuel costs. The challenge is to identify the optimal route, considering variables like road distance, traffic conditions, vehicle fuel efficiency, and user satisfaction

Task

Discuss how you can utilize MDP for route planning. Define states, actions, rewards, transitions. Optimize route selection

TRANSFER TASK
PRESENTATION OF THE RESULTS

Please present your results.

The results will be discussed in plenary.





1. A traveling salesman needs to visit 10 cities. A tour starts at a home city x = 1, visits each city exactly once, then returns to x[1]. The fifth city visited must be a specific city x = 5. The object is to determine which tour is the shortest, in total kilometers, under the given constraint. If the problem was to be solved by complete enumeration, how many tours would need to be evaluated?
   a) 5,040
   b) 14,400
   c) 40,320
   d) 362,880



2. The total reward for a two-stage sequential decision process is given by R = u(x[1]) + u(x[2]), where u(x) = sqrt(x) is a utility function; and x[1] and x[2] are nonnegative, and constrained to satisfy x[1]+x[2] = 50. What is the maximum total reward?
   a) 2 / 5
   b) 10
   c) sqrt(50)
   d) 2 * sqrt(50)



3. A simple game consists of three stages. Each stage k = 1,2,3 has a reward R(k), say R(1) = 20, R(2) = 20, R(3) = 30. Each stage k also has a quitting cost C(k), say C(1) = 5, C(2) = 35, C(3) = 60. At stage k, the player earns the current stage reward R(k), then decides whether or not to quit. If the player quits, he keeps all rewards earned so far, but deducts from that total the stage k quitting cost C(k). The game ends at that point. The player starts at Stage 1 and must quit by the end of Stage 3. What policy maximizes the player's total reward (net total cost)
   a) The optimal policy is to quit at Stage 1
   b) The optimal policy is to quit at Stage 2
   c) The optimal policy is to quit at Stage 3
   d) All policies yield the same reward

LIST OF SOURCES

Adda, J., Cooper, R., & Cooper, R. W. (2003). Dynamic economics: Quantitative methods and applications. MIT Press.

Bellman, R. (1957). Dynamic programming. Princeton University Press

Sutton, R. S., & Barto, A. G. (2018a). Reinforcement learning: An introduction (2nd ed.). MIT press.

Xue, S., Li, Z., & Yang, L. (2019). Training a model-free reinforcement learning controller for a 3-degree-of-freedom helicopter under multiple constraints. Measurement and Control, 0020294019847711

© 2022 IU Internationale Hochschule GmbH
This content is protected by copyright. All rights reserved.
This content may not be reproduced and/or electronically edited, duplicated, or distributed in any kind of form without written permission by the IU Internationale Hochschule GmbH.